

Assignment-1 (CS3530)

Multi-Client Quiz System with LLM Integration

Deadline: Sep 15, 2025, 11:59 PM

Penalty: 5% marks will be deducted for each day after the due date.

Individual Assignment

Programming Language: C/C++

The goal of this assignment is to develop your practical skills in client-server programming, concurrency management, and protocol design, while exploring the use of language models (LLMs) in real-time applications. You will build a system where multiple clients can connect to a central server, choose a quiz genre, receive dynamically generated questions from an LLM, submit answers, and receive scores. The abstract steps are given below:

Step 1: The client sends a genre (e.g., history, computers, etc) to the server

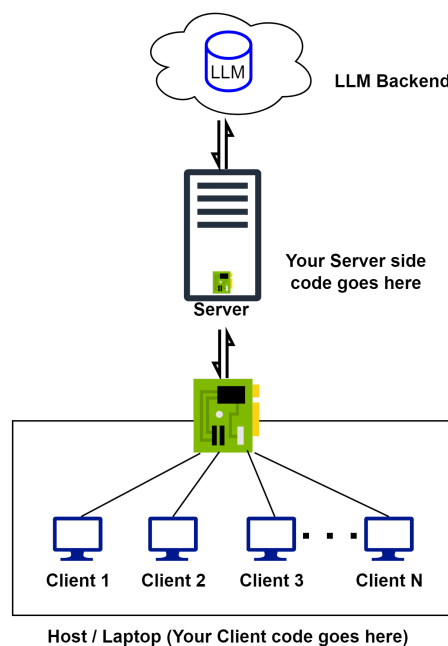
Step 2: The server constructs a prompt to be sent to the LLM backend to receive one MCQ based on the genre

Step 3: The server receives the MCQ (both question and answer) and relays only the question to the client.

Step 4: The client chooses an answer and sends the answer to the server.

Step 5: The server evaluates if the answer is correct/wrong and sends the appropriate response back to the client. Also, the server maintains the scores of the clients in a leaderboard.

Step 6: The client can also request the leaderboard at any point in time. The server should respond with the leaderboard details accordingly.



Your job will be to write the **server-side logic** and **client-side logic**. First, we will provide individual accounts on the server for you to develop and maintain your own server-side code base. Second, we will provide the LLM backend APIs, which you can call from your code. Further requirements about the server and client-side logic are given below.

Part 1: Client–Server System Implementation

A) Client-Server (without Cache)

(20 marks)

In this part of the assignment, you will design and implement a quiz application using a custom client–server protocol. The server must handle multiple concurrent clients, deliver quiz questions in real time, evaluate answers, and maintain a leaderboard.

System Description:

- The **server** maintains an open connection with each client during the quiz session (i.e. 5 minutes).
 - *Hint: Use multiple terminals and execute the client program*
- For each connected client:
 - The server generates quiz questions using **LLM API provided <To be shared with students>**.
 - The server only sends the **questions** (not the answers) to each client in real time.
 - The server receives and evaluates each submitted answer, updates client scores, and handles inactive clients using **KEEP-ALIVE messages**.
 - The server closes the connection after the time limit (i.e. 5 mins), and the client should display a message like “Quiz over – final scores available”.
- On request from the client, the server aggregates scores and returns a **leaderboard** showing scores of all the connected clients.
- The **client**:
 - Connects to the server over TCP,
 - Selects a quiz genre (e.g., Science, History, Sports),
 - Receives and displays questions,
 - Submits answers to the server,
 - Requests (from the server) and displays the leaderboard.

You **must design a custom application protocol** to differentiate between question messages, answer messages, and leaderboard requests. **<A template of protocol to be shared with students>**

Technical Requirements

1. Server Requirements

- Handle **multiple concurrent clients (Maximum 10 clients)** reliably.

- i. **Hint:** Use a multi-threading mechanism
 - Keep connections open until quiz completion and **stream questions** to each client.
 - The server should close the connection after quiz completion, and the client should display a message like “Quiz over – final scores available”.
 - **Score evaluation** and leaderboard generation on request.
 - Implement **KEEP-ALIVE** messages to detect dead clients :
 - i. The server must periodically send a **KEEP_ALIVE** message to each connected client (e.g., every 5 seconds).
 - ii. The client must reply promptly with an **ALIVE_OK** message.
 - iii. If the server does not receive a response within a specified window (e.g., 5 seconds), it must assume the client has disconnected and remove it from the active session and leaderboard.
 - iv. These messages must be part of your custom protocol and clearly documented in your protocol design document.
 - The server must print log messages (e.g. client 192.168.0.1:3000 connected) whenever applicable.
2. **Client Requirements:** Each client should have a **CLI-based interface** that allows the user to:
- Establish a connection and **select a quiz genre**.
 - Receive questions, display them to the user, and **send answers** using the custom protocol.
 - Request and display the **leaderboard**.
3. **Custom Protocol Design**
- Design and document message types (e.g., questions, answers, leaderboard) and header frame
 - Ensure messages are **well-structured** to distinguish between different operations.
 - Submit a short document describing your protocol format with examples.

B) Client-Server (with Cache)

(20 Marks)

In this part, you will extend your Part-1(A) implementation to incorporate **server-side caching**. The server must avoid redundant quiz generation requests by storing and reusing previously generated questions. This improves performance and demonstrates how caching works in real networked systems.

System Description:

- The server caches quiz questions (along with the answers) per genre to reduce repeated API calls.

- If a client requests a genre that is already cached, the server reuses cached questions instead of generating new ones.
- If a genre is not cached, the server fetches new questions (via prompt or simulated LLM API), stores them in the cache, and then serves them to the client.
- The client's behaviour is unchanged from Part 1.
- You must design your custom protocol to explicitly signal when cached data is used.

Technical Requirements:

1. Server Requirements

- Implement a **cache data structure** to store quiz questions and answers by genre.
- When a cached genre is requested, serve questions **directly from cache** (without sending a request to LLM).
- When a new genre is requested, request LLM to generate a question and an answer, and **update the cache** with the new questions and answers.
- Printed log messages must reflect whether a client was served using cached or fresh questions.
- **Invalidate or update cache entries** if needed (e.g., after a configurable time or question limit). Come up with your best cache invalidation policy with proper justification.
- Continue to handle **multiple concurrent clients, scorekeeping, and leaderboards** as in Part 1 (A).

2. Client Requirements

- Reuse your Part 1(A) client (minimal modifications).
- Display whether a question came from **cache or fresh generation** (based on server messages).
- Request and display the updated leaderboard as before.
- For testing of multiple clients, the client should automatically provide random responses to server queries (genre, answers to the questions, etc..)

3. Custom Protocol Extension

- Add message types or flags to indicate **cache hits vs. cache misses**
- Document these changes clearly with **example exchanges**

Part 2: Performance Measurement and Traffic Analysis

(60 Marks)

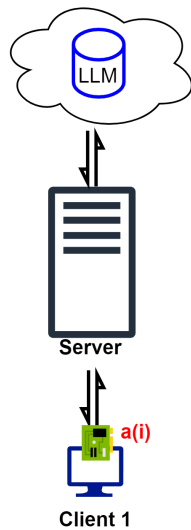


Fig 1. Single client

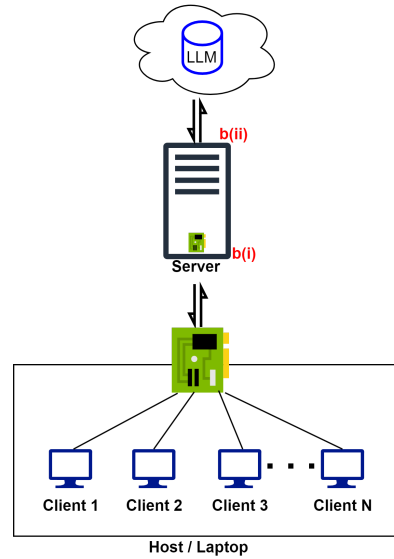


Fig 2. Multiple clients

1. Capture .pcap traces during quiz sessions for both 1(A) and 1(B) for different scenarios:
 - a. **Single client present:**
 - i. Capture at the **client host NIC** (close to the client).
 - b. **Multiple clients present (maximum 10 clients):** capture at **both server-side interfaces:**
 - i. **Server–Client NIC** (server interface facing the clients) (*Capture in your system*)
 - ii. **Server–LLM NIC** (server interface used for LLM/“simulated LLM” communication) **NOT NEEDED**
2. Analyse these from the client side: Throughput and latency (the time between request and response) under multiple concurrent clients. (i.e., 1, 2, 4, 6, 8 and 10 clients). Also, plot the graphs of throughput and latency against the number of concurrent clients.
 - a. Throughput: Effective throughput during the quiz (i.e., the rate at which quiz messages are exchanged between the client and server, in terms of bytes per second over TCP)
 - b. Latency:
 - i. Time taken from the quiz generation request to receiving the question(s)
 - ii. Time taken from submitting answer(s) to receiving the score
3. Measure system performance as the number of clients increases:
4. Identify bottlenecks (network vs server vs LLM latency).
 - a. What is the threshold of the LLM (How many requests can the LLM handle per second)?
 - b. How many concurrent clients is your server able to handle?

Deliverables (detailed):

1. **Source code** of both server and client with clear instructions to compile and run (for both scenarios 1(A) and 1(B))
2. **Report that includes:**
 - a. **Protocol design document** (1–2 pages):
 - i. Message types and format
 - ii. Example client–server exchanges
 - b. **Testing evidence:**
 - i. Screenshots/console logs showing:
 1. At least 3 concurrent clients connected
 2. Clients receiving questions, submitting answers
 3. Server scoring & leaderboard output
 - ii. Provide one set for **1(A)** and one set for **1(B)**
 - c. **Performance Measurement and Traffic Analysis:** Provide a concise report section with the following structure. Use figures/tables for clarity.
 - i. Collect and analyse network traces and performance results for **both** 1(A) and 1(B) under the scenarios below. Your answers **must** be justified using **.pcap** evidence.
 - ii. Collect **separate** trace files per run.
 1. Limit your packet capture to the quiz traffic only (use a port filter) and no longer than 30 seconds per run.
 2. If it is required to run longer tests, submit **screenshots/CSV outputs only** — no full trace submission.
 3. Include capture commands (e.g., `tcpdump -i <iface> -s 0 -w <file>.pcap 'port <PORT>'`) in the README.
 - iii. Metrics & Methodology
 1. **Latency:** define clearly (e.g., question latency = time from server sending QSTN to server receiving corresponding ANSR; or network RTT between client↔server measured from SYN/ACKs or app messages).
 2. **Throughput:** define units (e.g., bytes/sec) and measurement window.
 3. **Concurrency sweep:** run with N (clients) $\in \{1, 3, 5, 7, 10\}$ and study latency, throughput, timeouts, packet loss, and CPU usage.
 - a. For each N :
 - i. Start the server.
 - ii. Launch N client instances at roughly the same time.
 - iii. Capture performance metrics: latency, throughput, timeouts, packet loss, CPU usage, etc.
 - b. Plot or tabulate results (wherever required): For example, latency vs. N , throughput vs. etc.

4. **How traces were analysed:** Wireshark filters or scripts (attach parsing scripts if used).

iv. Bottleneck Identification:

1. Answer with evidence (referencing specific packets/flows in .pcaps):

a. Is the bottleneck the network, server CPU, or LLM latency?

b. Use indicators like increasing server–LLM request service time, queueing at server (delayed sends), TCP retransmissions, high RTTs, or CPU saturation (top/vmstat outputs may be appended).

v. Server Concurrency Limit:

1. How many concurrent clients can your server handle before:

2. Errors, timeouts, or unacceptable latency occur?

3. **README file** with:

a. Setup steps

i. OS/compiler version (or interpreter)

ii. Installation of any dependencies in one or two lines.

b. How to run the server and clients:

i. Exact command(s) for both 1(A) and 1(B) (no guesswork).

ii. Default port/timeout/number of questions (if hardcoded).

c. Expected input/output format:

i. Show a sample run for both server and client (e.g., “Server prints Waiting for clients..., client prints Question: ...”).

ii. Show how leaderboard output looks.

4. Deliverables checklist (summary):

a. Source code (server/client) for 1(A) and 1(B)

b. README with compile/run steps + sample run + expected output

c. .pcap files for all test scenarios

d. Report including:

i. Protocol design doc

ii. Screenshots/logs for functionality tests (3+ clients),

iii. Performance analysis (tables, graphs, bottleneck findings)

Note:

- The marks will be awarded based on your performance in the viva.
- TAs will not be available for any help on the last 2 days of the deadline. So make sure that you reach out to TAs on time if you need any help with setup and configuration.
- The deadline will not be extended even if the LLM crashes in the last 2 days.